

Module 04: Feature Engineering II - Target Encoding and the Power of Smoothing

Cybernauts 2026 Datathon Campaign

May 2026

Abstract

Target Encoding is arguably the most powerful encoding technique in a data scientist's arsenal, especially for high-cardinality features like `ZIP_Code` or `Product_ID`. However, it is a "double-edged sword." If used without caution, it introduces **Data Leakage** and causes your model to overfit instantly. This note covers how to implement Target Encoding safely using M-Estimate Smoothing.

Contents

1	The Core Concept: Encoding the Signal	2
2	The Danger: Data Leakage and Overfitting	2
3	The Solution: M-Estimate Smoothing	2
3.1	The Smoothing Formula	2
3.2	How Smoothing Behaves	2
4	Python Implementation	2
5	Practice Problems	3

1 The Core Concept: Encoding the Signal

In earlier modules, we discussed One-Hot and Frequency encoding. **Target Encoding** goes a step further by using the information from the target variable (y) itself to encode the categories.

Logic: You replace each text category with the **average target value** for that specific category.

Example: If you are predicting "Loan Default" (0 or 1) and users in "Dhaka" default 15% of the time, the category "Dhaka" is replaced by the number **0.15**.

2 The Danger: Data Leakage and Overfitting

The biggest risk of Target Encoding is **Overfitting**.

The Single-Row Trap

Imagine a category called "Specific_Device" that only appears **once** in your entire dataset. If that specific user happened to default (Target = 1), the encoded value for that category becomes exactly **1.0**. The model will see this 1.0 and think it has found a "perfect predictor," but in reality, it has just memorized a single row. This is a form of data leakage.

3 The Solution: M-Estimate Smoothing

To fix the overfitting problem, we don't use the raw category average. Instead, we "smooth" it by blending it with the **Global Average** of the entire dataset.

3.1 The Smoothing Formula

The encoded value S_i for a category i is calculated as:

$$S_i = \alpha_i \cdot \bar{y}_i + (1 - \alpha_i) \cdot \bar{y}_{\text{global}} \quad (1)$$

Where:

- $\bar{y}_i$: The average target value for the specific category.
- $\bar{y}_{\text{global}}$: The average target value for the **entire** training set.
- α_i : A weight (between 0 and 1) based on the number of samples in that category.

3.2 How Smoothing Behaves

- **High Count Categories:** If a category appears 10,000 times, α_i is close to 1. The encoded value stays very close to the specific category average ($\bar{y}_i$).
- **Low Count Categories (Rare):** If a category appears only 2 times, α_i is close to 0. The encoded value is "pulled" toward the global average ($\bar{y}_{\text{global}}$), preventing the model from overreacting to a tiny sample size.

4 Python Implementation

While you can calculate this manually, the `category_encoders` library is the industry standard for safe implementation.

```
1 from category_encoders import TargetEncoder
2
3 # Initialize encoder with smoothing parameter
4 te = TargetEncoder(smoothing=10)
5
6 # Fit and transform (IMPORTANT: Only fit on Training data to avoid leakage!)
7 X_train['City_Target'] = te.fit_transform(X_train['City'], y_train)
```

```
8 X_test['City_Target'] = te.transform(X_test['City'])
9
```

5 Practice Problems

P1: Why is it critical to only fit the TargetEncoder on the training set and not the full dataset?

*Answer: To prevent **Data Leakage**. If you use the test set targets to calculate the encoding, the model "sees" the future during training and will give an artificially high accuracy that fails on real data.*

P2: If the Global Average Default Rate is 0.10, and a rare city "Khulna" appears only twice with a 0.50 default rate. Will a smoothed Target Encoder output 0.50?

Answer: No. Because it is a rare category, the encoder will pull the value down toward the global average (0.10), resulting in a value like 0.15 or 0.20 instead of 0.50.